

A Client-Side Architecture for Scalable Ingestion and Interactive Visualization of Cosmological Particle Data

Idunnuoluwa A. Adeniji^{1,2}, Joseph Insley², Michael E. Papka^{1,2}
University of Illinois Chicago¹, Argonne Leadership Computing Facility, Argonne National Laboratory²

Email: laden@uic.edu
insley@anl.gov
papka@uic.edu

OPENCOSMO VISUALIZATION APPLICATION (OVA)

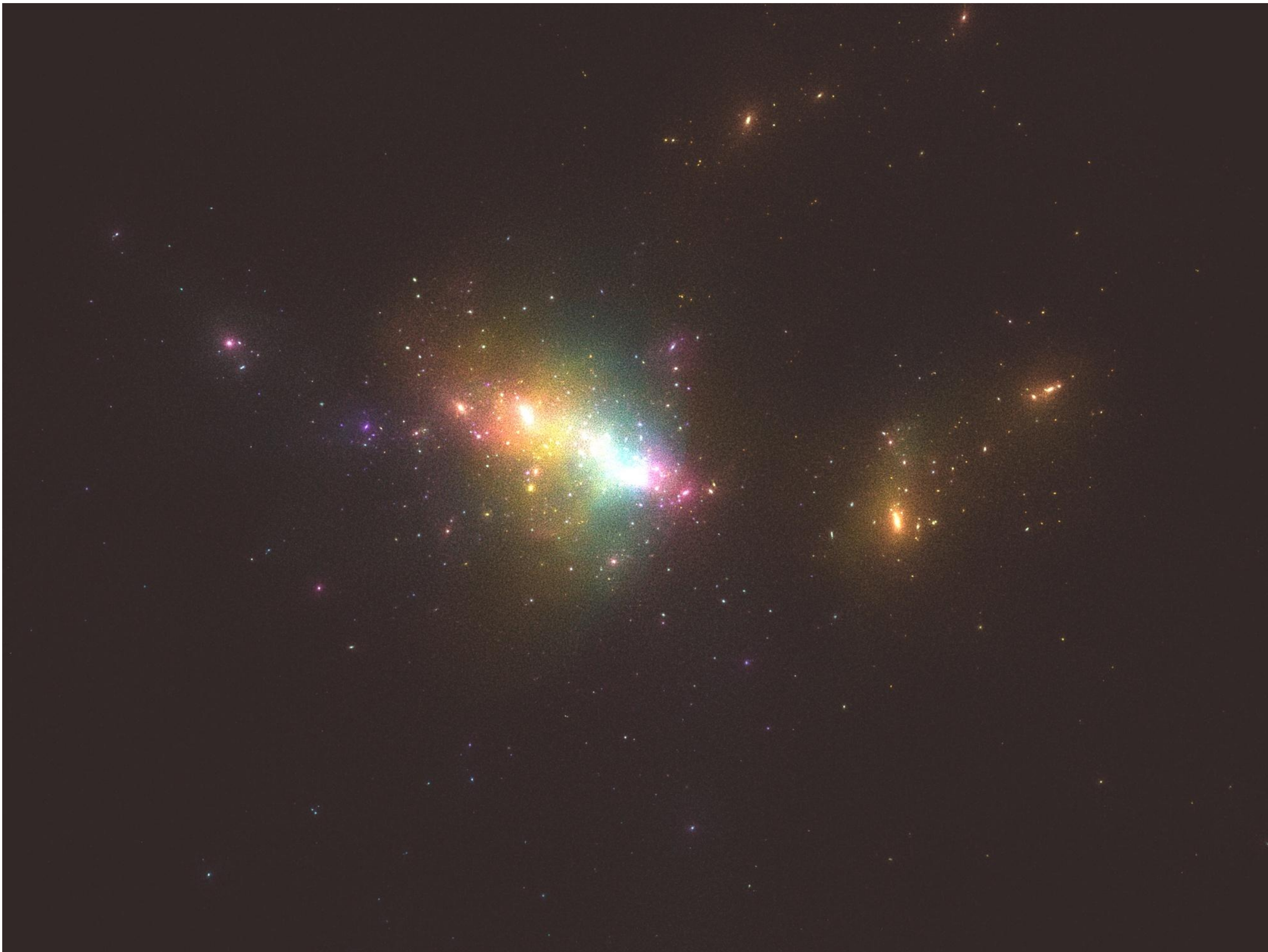
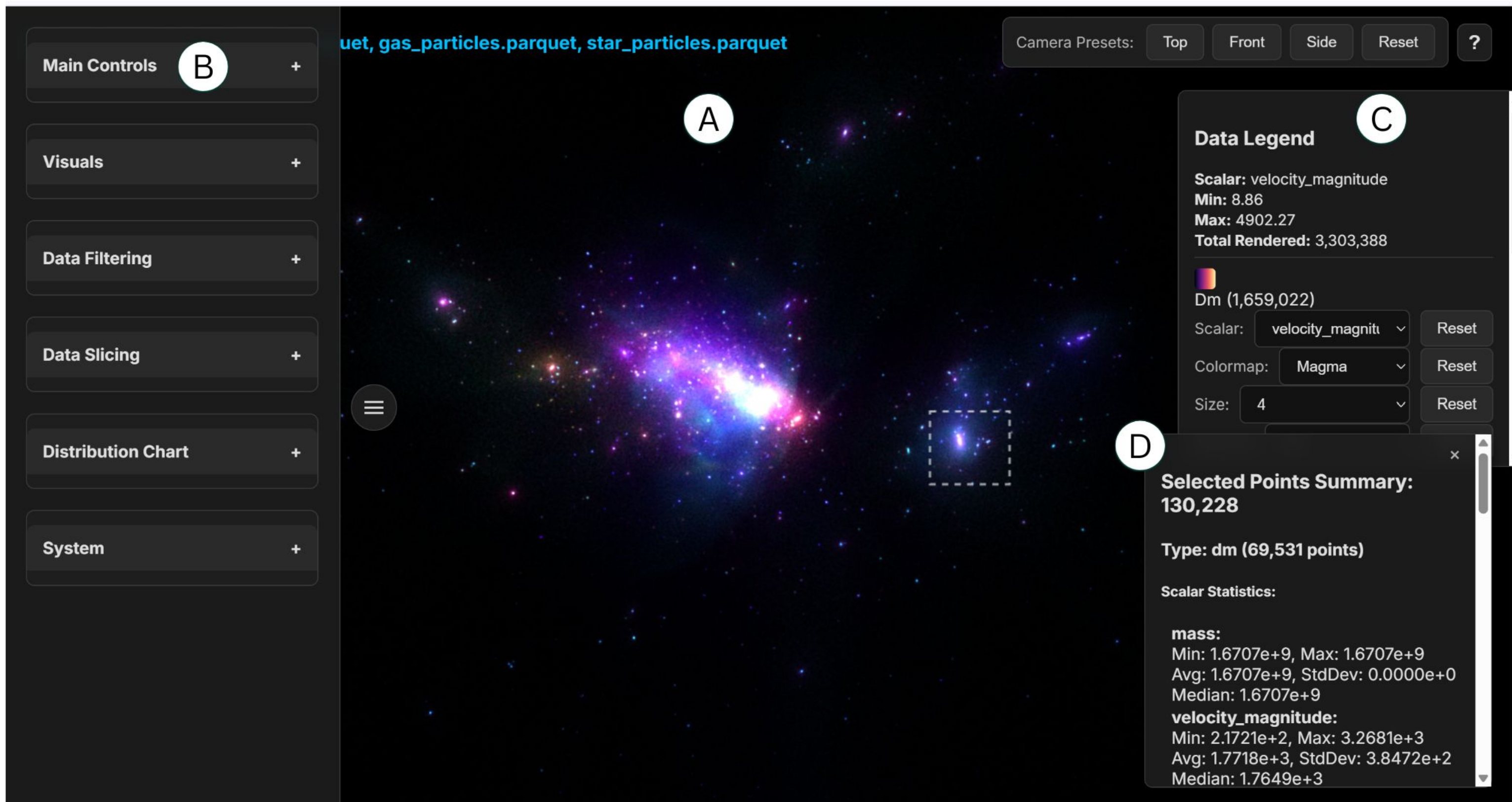


Figure 1. Visualization of a cosmological particle dataset produced with the OpenCosmo Visualization Application.

MOTIVATION & SYSTEM CONTRIBUTIONS

- **OpenCosmo portal** delivers millions of simulation particles with no integrated visualization — users must export to external tools, fragmenting workflows
- **OVA** enables **fully client-side ingestion** and **GPU-accelerated rendering** of Apache Parquet datasets — no preprocessing, no server-side dependencies
- Columnar Transferable buffer architecture: **13× faster transfer**, **4.3× less heap**, extending capacity from **16.5M** → **52.9M particles**
- Zero long main-thread tasks (>50 ms) across all tested scales — main thread stays responsive during large dataset loads
- **≥49.5 FPS GPU-accelerated rendering** on commodity hardware — integrated GPU, no dedicated visualization hardware required



OVA interface: (A) 3D particle rendering, (B) control panel, (C) per-type data legend, (D) interactive selection summary.

RELATED WORK

Property	Firefly	BabiaXR	PVWeb	Ours
No preprocessing	X	✓	X	✓
Parquet input	X	X	✓*	✓
Client-side only	✓	✓	X	✓
Derived scalars	X	—	✓*	✓
Per-type controls	Partial	X	✓	✓
GPU 3D clipping	X	X	✓	✓
WebXR / VR	X	✓	X	✓
Portal embedded	X	X	Partial	✓

*Server-side only; requires dedicated backend infrastructure.

Table 1. Comparison across key properties relevant to portal-embedded cosmological data exploration.

METHODS

- **Ingestion (WASM + Apache Parquet)**
 - Parquet files ingested in-browser via parquet-wasm + Apache Arrow — no server preprocessing. Velocity magnitude derived in the worker before transfer.
- **Rendering (WebGL2 + Babylon.js)**
 - GPU mesh uploaded once — all interaction via shader uniforms, no re-upload
 - Additive blending + Gaussian profiles + Reinhard tonemapping
- **Bottleneck & Fix**
 - Structured-clone: 14.5 s at 3.3M, OOM at 16.5M
 - Fix: columnar Float32Array + Transferable → 1.09 s, constant time
 - 13× faster · 4.3× less heap · zero long tasks

SYSTEM ARCHITECTURE

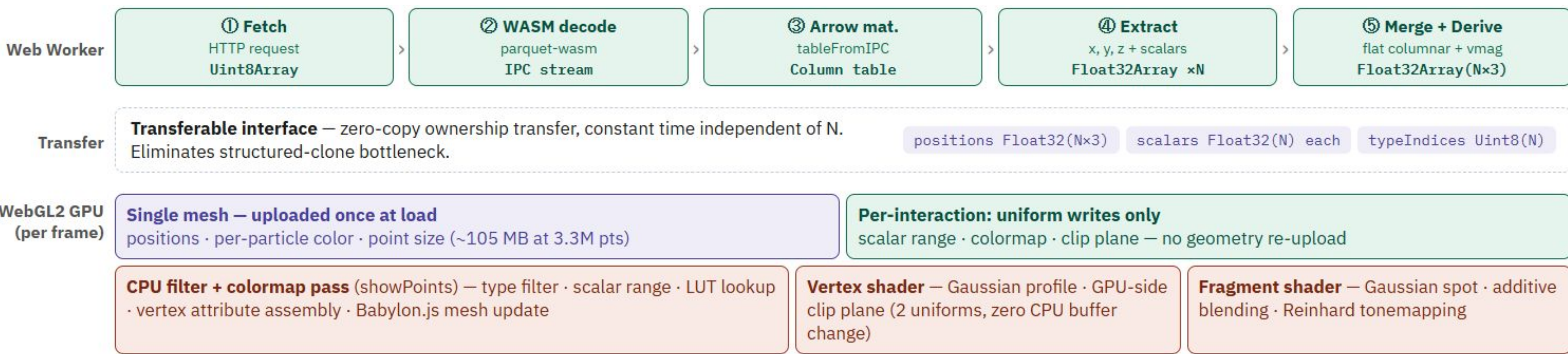


Figure 2. Diagram of OVA's System Architecture

RESULTS

13×
faster transfer

4.3×
less heap

52.9M
max particles

0
long tasks (>50ms)

≥49.5
FPS GPU-accel.

TRANSFER BENCHMARK (3.3M PARTICLES, AVG 3 RUNS)

	Transfer (ms)	Heap (MB)
Structured-clone (baseline)	14,494	1,860
Transferable (ours)	1,092	436

SCALING BENCHMARK (TRANSFERABLE ARCHITECTURE, AVG 3 RUNS EACH)

Particles	Pipeline (ms)	Transfer (ms)	Rendered	FPS	Heap (MB)
3.3M	2,940	1,092	3.3M	55.6	436
16.5M	13,905	1,471	5.0M†	52.0	828
26.4M	23,807	875	5.0M†	60.0	1,147
36.3M	32,048	1,015	5.0M†	60.0	1,553
42.9M	36,101	1,451	5.0M†	60.0	1,823
52.9M	43,977	1,325	5.0M†	49.5	2,418

†Rendered pts capped at 5M via random sampling. Pipeline scales at ~0.83 ms per 1,000 particles. Transfer <1.5 s at all scales — constant-time Transferable confirmed. At 52.9M, heap (2.4 GB) stays within Chrome's ~4 GB per-tab limit.

†Rendered pts capped at 5M via random sampling. Pipeline scales at ~0.83 ms per 1,000 particles. Transfer <1.5 s at all scales — constant-time Transferable confirmed. At 52.9M, heap (2.4 GB) stays within Chrome's ~4 GB per-tab limit.

*Benchmarks: Windows 11 (build 26100), 16 GB RAM, Chrome 145. WebGL2 via ANGLE (D3D11) on AMD Radeon 780M (RDNA 3, integrated GPU). RTX 4060 present but unused — results represent mid-range integrated GPU performance.

DISCUSSION

- Each additional user adds zero marginal HPC cost — server only hosts static files
- At 52.9M particles, heap (2.4 GB) remains within Chrome's ~4 GB per-tab limit
- Client hardware variability is the key tradeoff — OVA targets rapid first-look exploration before export to ParaView or VisIt

FUTURE WORK

- **WebGPU + compute shaders** (In progress) — GPU-resident filtering eliminates CPU rebuild; enables full-dataset LOD
- **Portal integration** (In progress) — launch directly from OpenCosmo query results
- **Brushing, linking & collaborative sessions** — multi-view analysis and shared exploration

REFERENCES

1. S. Habib et al., New Astronomy, vol. 42, 2016.
2. Argonne National Laboratory, cosmoexplorer.alcf.anl.gov
3. A. B. Gurvich & A. M. Geller, ApJS, vol. 265, no. 2, 2023.
4. D. Moreno-Lumbreras et al., SoftwareX, vol. 24, 2023.
5. S. Jourdain et al., IJCISIM, vol. 3, 2011.
6. M. J. Turk et al., ApJS, vol. 192, no. 1, 2011.
7. D. Vohra, Practical Hadoop Ecosystem, Apress, 2016.
8. Babylon.js, doc.babylonjs.com